P1351R0
Intrusive smart pointer feedback

Published Proposal, 2019-01-20

This version:
http://wg21.link/P1351R0

Author:
Mark Zeren (VMware)

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:
https://github.com/mzeren-vmw/P1351

Abstract

Provide feedback for [P0468R1] based on experience with intrusive smart pointers at VMware.

Table of Contents

§ 1. Introduction

This rather hastily assembled document provides feedback for [P0468R1] based on experience with intrusive smart pointers at VMware. In summary:

- **Less overhead is an objective** - Intrusive pointers offer significantly less overhead than `std::shared_ptr`. This objective should inform the design for the Standard Library.
- **Raw pointers are a valid use case** - For intrusively reference counted types, raw pointer parameters are inherently less expensive than passing a smart pointer by reference or value.
- **Intrusive smart pointers should retain by default** - Retain by default behavior follows from the raw pointer use cases. [P1132R2]'s `out_ptr` covers the C interface use case where we want adopt by default behavior.

§ 2. Less Overhead

`std::shared_ptr` provides support for features which, while useful, incur compile time and runtime overhead. Intrusive smart pointers have fewer features and, correspondingly, less overhead.

[P0468R1] focuses on interoperation with C interfaces, but providing users with a lower cost alternative to `std::shared_ptr` is an equally important motivation.

The following sections describe some of the costs incurred by `std::shared_ptr`.

§ 2.1. Inner pointers

`std::shared_ptr` provides support for being rebound to point to a sub-object. This feature forces `std::shared_ptr` to *always* hold two pointers. Intrusive smart pointers, on the other hand, can be implemented in terms of a single pointer.

§ 2.2. Type Erasure and Deleters

`std::shared_ptr` holds a type erased deleter, introducing compile time and run time overhead.

§ 2.3. `std::weak_ptr`

`std::shared_ptr` must *always* provide additional shared state storage and runtime logic to support `std::weak_ptr`, even if an application never uses it. Intrusive smart pointers, on the other hand, do not have support for weak pointers.

§ 2.4. Allocation of shared state

`std::shared_ptr` must be responsible for allocating shared state. The runtime cost of this allocation can be effectively reduced or eliminated via `std::make_shared` which combines shared state allocation with controlled object allocation. Nonetheless, managing allocations increases the complexity of `std::shared_ptr`. Intrusive smart pointers avoid this complexity by deferring allocation to the pointed-to type.

§ 3. Raw Pointers

Sample code for the following sections can be found in this godbolt.org playground: Godbolt.org

§ 3.1. Raw Pointer Parameters

Passing a pointer generates better code than passing a smart pointer by reference.

Say we have:

```
#include <cstdio>

using namespace std;

struct Scout {
  virtual const char* getName() = 0;
  ...
};
```

And a function that takes a pointer:

```
void greet(Scout* scout)
{
  printf("Hello %s", scout->getName());
}
```

Which will generate the following code (with `-Os`, but the argument holds for `-O3` as well):

```
1 greet(Scout*):
2         subq    $8, %rsp
3         movq    (%rdi), %rax
4         call    *(%rax)
5         popq    %rdx
6         movq    %rax, %rsi
7         movl    $.LC0, %edi
8         xorl    %eax, %eax
9         jmp     printf
```

Now look at a function that takes a `const retain_ptr&`:

```
void greet(const retain_ptr<Scout>& scout)
{
  printf("Hello %s", scout->getName());
}
```

It will generate:

```
 1 greet(const retain_ptr<Scout>&):
 2         subq    $8, %rsp
 3         movq    (%rdi), %rdi   <--- HERE
 4         movq    (%rdi), %rax
 5         call    *(%rax)
 6         popq    %rdx
 7         movq    %rax, %rsi
 8         movl    $.LC0, %edi
 9         xorl    %eax, %eax
10         jmp     printf
```

Look at line 3. We have an additional indirect load.

This might seem like a small thing, but it will add up in a large codebase.

§ 3.2. Extrapolating from there

If we should always pass by pointer, then getters should return by pointer too. Otherwise, we have to sprinkle code with verbose `.get()`s:

```
struct Expedition {
  Scout* getScout();
};

void start(Expedition& journey)
{
  greet(journey.getScout());  // As opposed to getScout().get().
}
```

Of course the `Scout*`'s lifetime is scoped to `journey`'s lifetime. We expect that in the future this semantic will be enforceable by static lifetime checkers. See [LIFETIME].

§ 4. Retain by Default

If we traffic in bare pointers with transitive ownership semantics (because it generates better code), assignment to a smart pointer indicates intent to add a new shared owner. In this case the smart pointer should retain by default.

§ 4.1. Retaining arguments

When we want to retain a result or a passed in argument (of type `T*`) `operator=` is the natural tool to use.

```
struct Expedition {
  ...
  void setScout(Scout* scout) { scout_ = scout; }   // operator=
  ...
};

static retain_ptr<Scout> sCave;
void exploreCave(Expedition& e)
{
  sCave = e.getScout();                             // operator=
  e.setScout(nullptr);
}
```

§ 4.2. `boost::intrusive_ptr`

`boost::intrusive_ptr` has a retaining `operator=(T *)`.

§ 4.3. `adopt` / `release` still required

Of course we still need the ability to "adopt" and [P1132R2]'s `out_ptr` should adopt by default for intrusive smart pointers.

```
extern void C_Recruit(Scout**);

retain_ptr<Scout> recruit()
{
  retain_ptr<Scout> scout;
  C_Recruit(std::out_ptr(scout));
  return scout;
}
```

Adoption is typically only used at the interface with "C" APIs, and should be less frequent than parameter passing and result returning.

§ 5. Conclusion

We should add intrusive smart pointers to the Standard Library not only to interoperate with C interfaces, but also so that our users do not have to pay for the features of `std::shared_ptr` that they do not use. Bare pointers are a natural, and less expensive, parameter and return value type for intrusively reference counted types. It follows that assigning a bare pointer into a smart pointer should retain by default. However, `std::out_ptr`, which operates at the C interface boundary should adopt by default.

§ References

§ Informative References

[LIFETIME]

Lifetime safety: Preventing common dangling. 25 September 2018. URL: https://github.com/isocpp/CppCoreGuidelines/blob/master/docs/Lifetime.pdf

[P0468R1]

Isabella Muerte. An Intrusive Smart Pointer. 19 June 2018. URL: https://wg21.link/p0468r1

[P1132R2]

JeanHeyd Meneide, Todor Buyukliev, Isabella Muerte. out_ptr - a scalable output pointer abstraction. 26 November 2018. URL: https://wg21.link/p1132r2